Machine Learning
Semester 2026-III
Workshop No. 1 — Data Preparation & Supervised Learning Baselines

Eng. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the first workshop of the *Machine Learning* course project!

This workshop marks the beginning of your semester-long project. Before implementing advanced models, your team must deeply understand your assigned dataset, establish a clean and reproducible data pipeline, and produce baseline models that will serve as reference points for everything that follows.

The central question for this workshop is: **What does your data look like, and what performance can you achieve with well-engineered baseline models?**

Your answers will establish the difficulty of the problem, the degree of class imbalance, data quality characteristics, and baseline performance metrics that serve as reference points for comparative analysis.

General Workshop Definition: Data preparation and supervised learning baselines involve downloading, validating, preprocessing, and characterising the assigned dataset, then establishing rigorous baseline models using traditional machine learning techniques applied to appropriate feature representations. The goal is complete mastery of the data, its structure, and its challenges before proceeding to more complex approaches.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

Workshop Scope and Objectives

- **Dataset Download and Integrity Verification:** Download the assigned dataset, verify data integrity, and document all access steps reproducibly.
- **Exploratory Data Analysis (`EDA`):** Characterise the dataset through distribution analysis, statistical summaries, quality assessment, and visual inspection appropriate to your data type.
- **Data Preprocessing Pipeline:** Implement a reproducible preprocessing pipeline covering data cleaning, normalization, feature engineering, augmentation strategy, and train/validation/test split.
- **Supervised Baseline Models:** Train at least three traditional machine learning classifiers on engineered features and report their performance using appropriate evaluation metrics for your task.
- **Repository Initialisation:** Set up the course repository with the required folder structure and a comprehensive `README.md`.

Dataset Access and Acquisition

Each group has been assigned a specific dataset for your domain. **All datasets are freely available** — no paid subscriptions are required. Your instructor has provided dataset-specific information separately. General principles for dataset acquisition:

1. **Verify Dataset Legitimacy and License:**
   - Confirm you have legal access to the dataset with appropriate licenses and permissions
   - Read and understand any usage terms, restrictions, or citations required by the dataset provider
   - Verify dataset size and storage requirements on your system
2. **Download and Setup:**
   - Document all download procedures in your `README.md` for reproducibility
   - Verify file integrity if checksums are provided
   - Organize files in a consistent directory structure (e.g., `data/raw/`, `data/processed/`)
3. **Understand Your Data Format:**
   - Identify what types of files your dataset contains (images, CSV, JSON, time-series, text, etc.)
   - Understand the label/target format and class definitions
   - Note any metadata, annotations, or auxiliary files

- Document the task: classification, regression, structured prediction, etc.

Required Artifacts and Methodology

1. **Dataset Download and Integrity Verification:**
   - Download the complete assigned dataset and document all steps in the `README.md` so a new team member can reproduce the setup from scratch
   - Verify file counts and `MD5`/`SHA256` checksums where the dataset provider supplies them
   - Document the total dataset size on disk, number of samples per class, and data format characteristics
   - Write a `dataset_info.py` script that scans the dataset directory and prints a structured summary (sample counts per class, feature ranges, data types)
2. **Exploratory Data Analysis (`EDA`):**
   - *Target Distribution:* Analyze the distribution of targets/labels. For classification, produce bar charts of class counts. Compute imbalance ratios. Document data balance characteristics.
   - *Feature Statistics:* Compute summary statistics (mean, std, min, max) for numerical features. Identify data types and ranges appropriate to your domain.
   - *Data Quality:* Identify missing values, outliers, corrupted entries, or inconsistencies. Document percentage of usable vs. problematic samples.
   - *Feature Relationships:* Explore correlations, distributions, and patterns in your features. Identify which features may be predictive of your target.
   - *Visual Inspection:* Create visualizations appropriate to your data type (histograms for numerical data, examples from each class, heatmaps for correlations, etc.). Identify any surprising patterns or quality issues.
   - Document all findings in a structured `EDA` notebook (`notebooks/eda.ipynb`)
3. **Data Preprocessing Pipeline:**
   - *Format Standardization:* Ensure all data samples are in a consistent format. For images, resize to a standard resolution justified by your use case. For other data types, standardize units, scales, or representations.
   - *Normalization/Scaling:* Apply appropriate scaling or normalization to numerical features (e.g., z-score normalization, min-max scaling). Document which method was used and why.
   - *Missing Data Handling:* Address missing values through imputation, removal, or flagging, as appropriate for your domain. Document your strategy.
   - *Train/Validation/Test Split:* Define and document your train/validation/test split (recommended 70%/15%/15

- *Augmentation Strategy (Training Set Only):* Define data augmentation techniques appropriate to your domain (e.g., rotation, flipping, noise addition, etc.). Justify choices and apply augmentations only to the training set.
- Implement the pipeline in reproducible code (e.g., `src/dataset.py`) with clear documentation.

4. **Supervised Baseline Models:**

   Train at least three traditional machine learning baseline classifiers. The goal is to establish baseline performance before pursuing more complex approaches:

   - *Feature Preparation:* Engineer or select features appropriate to your data type. For tabular data, use domain-relevant features. For image data, extract handcrafted features (e.g., HOG, texture descriptors). Reduce dimensionality if necessary (e.g., with PCA).
   - *Baseline 1:* Train a linear model such as Logistic Regression or Linear SVM on the prepared features.
   - *Baseline 2:* Train a tree-based ensemble such as Random Forest.
   - *Baseline 3:* Train a third baseline of your choice (e.g., Naive Bayes, k-NN, Gradient Boosting).
   - *Evaluation:* For each classifier, report appropriate evaluation metrics for your task type:
     - *Classification:* accuracy, precision, recall, F1-score, and `AUC-ROC` (for binary or one-vs-rest).
     - *Regression:* mean absolute error, mean squared error, R^2 score.

     Present confusion matrices (for classification) or residual analysis (for regression). Use `sklearn.metrics`.
   - Document which metrics are most important for your task and why.

Deliverables and Documentation

1. **Data Preparation and Baseline Report (`PDF`):**

   - *Executive Summary*: Overview of the dataset and key findings
   - *Dataset Description*: Source, access procedure, dataset statistics and characteristics
   - *Exploratory Data Analysis*: Summary of EDA findings, data distributions, quality issues identified
   - *Preprocessing Pipeline*: Descriptions of preprocessing decisions and their justifications
   - *Baseline Model Results*: Table comparing performance of all three baseline classifiers on relevant metrics

- *Discussion*: What do the baseline results reveal about the problem difficulty, class balance, and features?

2. **Code:**

   - `src/data_loader.py` or `src/dataset.py` — Data loading and preprocessing code
   - `src/features.py` — Feature engineering/extraction code (if applicable)
   - `src/baselines.py` — Training and evaluation of the three baseline models
   - `notebooks/eda.ipynb` — Complete EDA notebook with all visualizations and analysis
   - `data/splits.json` — Train/validation/test index assignments for reproducibility

3. **Repository Initialisation:**

   - `src/` — all source modules
   - `notebooks/` — Jupyter notebooks for exploration and analysis
   - `data/` — split files and metadata (raw data should be accessed/downloaded separately)
   - `checkpoints/` — saved model weights (future workshops)
   - `runs/` — training logs (`wandb` / `TensorBoard`)
   - `README.md` — dataset download steps, environment setup, and commands to reproduce all results

Submission Requirements

- The `PDF` report must follow the naming convention: `groupN_workshop1.pdf` (e.g., `group3_workshop1.pdf`)
- Submit through the designated course platform by the specified deadline
- Include the `GitHub` repository link with all code and the `EDA` notebook
- All documentation must be in **English** and follow professional technical writing standards
- All reported metric values must be reproducible by running the submitted code
- Focus on traditional machine learning approaches and hand-crafted or domain-specific features in this workshop. Deep learning will be explored in later workshops.

Grading Note: 40% of the workshop grade will be determined by an in-class quiz administered during the course. This quiz will assess your team's understanding of data preparation, exploratory data analysis, baseline model development, and key findings from your workshop. The quiz emphasizes mastery of concepts and the ability to explain your methodology and results.

Important Guidelines and Considerations

- **Class/Label Imbalance:** Many real-world datasets have imbalanced target distributions. Never use accuracy as the sole evaluation metric for imbalanced data. Instead, report precision, recall, F1-score, and `AUC-ROC`. Understand what false positives and false negatives mean in your domain.
- **Data Leakage Prevention:** Compute all statistics, normalization parameters, and feature transformations *exclusively* on the training set. Apply these computed values to validation and test sets. Any information leaking from validation/test into training parameters will artificially inflate results.
- **Reproducibility First:** Set random seeds for `numpy`, `Python`, and `sklearn` at the start of every script. Save the train/validation/test split to `data/splits.json` and load it consistently in all subsequent work — never re-split randomly.
- **Understand Your Domain:** Read background literature and papers about your problem domain and dataset. Understand what the data represents, what it's measuring, and what patterns domain experts would expect to find.

- **Baseline Quality is Critical:** The baseline models and metrics you establish here serve as essential reference points. They reveal the true difficulty of your problem and provide anchors for all future comparisons. Invest time in getting baselines right.

Good luck, and start your machine learning journey the right way — by understanding the data before building any complex model. Every great machine learning system begins with careful exploratory analysis and rigorous baseline comparisons built on a reproducible foundation!